

Проектирование интеграций: от классики до AI-стандартов

AI Архитектор



Проверить, идет ли запись

Меня хорошо видно & слышно?





Иван Четвериков

AI Architect

- Более 5 лет опыта в разработке и архитектуре
- Специалист в разработке и проектировании AI-приложений в Raft

 @istrebitel_1

 istrebitel.3.12@gmail.com



Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Карта курса



СТАРТ

Стратегический фундамент
и планирование проекта

Проектирование и
документирование архитектуры

Продвинутые
архитектурные
паттерны

Инфраструктура

Качество, интеграции
и безопасность

Стратегия, лидерство и
экономика

Проектная работа

ФИНИШ



Маршрут вебинара

Знакомство

Синхронные протоколы и паттерны

Асинхронные протоколы и
паттерны

Практический кейс

AI-стандарты и новые протоколы

Основные тезисы



Цели вебинара

К концу занятия вы сможете

Смысл

Для чего вам это уметь

выбирать подходящие паттерны интеграции (API Gateway, Message Broker, ETL) для различных сценариев

проектировать обмен данными между AI-компонентами и традиционными системами

для того, чтобы AI компоненты не потеряли связь с окружающим миром



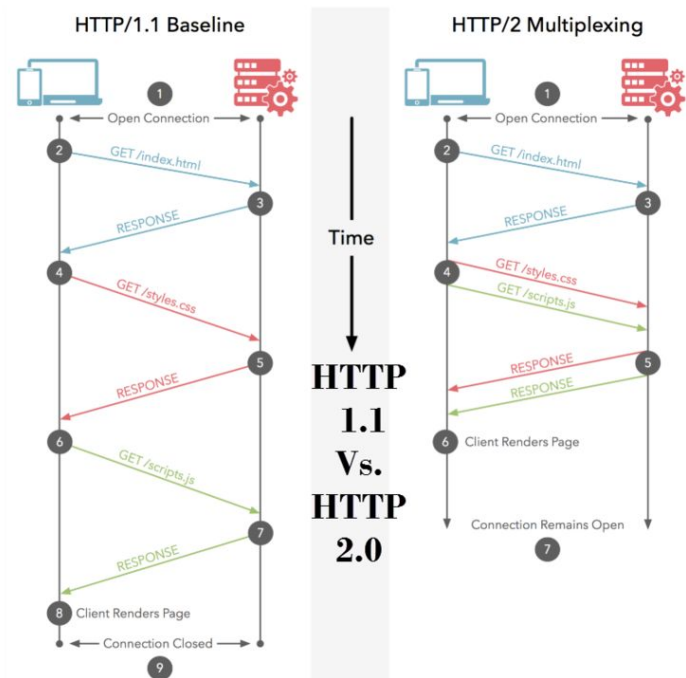
**Занимались ли Вы
интеграциями?**



Синхронные протоколы и паттерны

От HTTP/1.1 к HTTP/2 и далее

- HTTP/1.1: Текстовый, простой.
 - Проблема: Head-of-Line Blocking. Если один запрос завис, остальные ждут в очереди (в рамках одного соединения).
- HTTP/2: Бинарный, Мультиплексирование.
 - Позволяет слать сотни запросов параллельно в одном TCP-соединении.
 - Фундамент для gRPC: Именно HTTP/2 позволил реализовать эффективный двунаправленный стриминг.
- HTTP/3 (QUIC): Работает поверх UDP. Решает проблему потерь пакетов в нестабильных сетях (мобильные клиенты).



SMTP и специфические протоколы

- Сценарий: Интеграция с банковским Mainframe или старой ERP.
 - Проблема: Система не имеет API, работает за жестким фаерволом, но умеет слать Email.
- SMTP (Email as a Bus)
 - Используется как асинхронная очередь сообщений.
 - Плюсы: Проходит через любые корпоративные фаерволы, встроенные retry-механизмы почтовых серверов
- FTP/SFTP (File Exchange)
 - Паттерн "Hot Folder": AI-сервис следит за папкой. Появился файл -> началась обработка.
 - Идеально для Batch Processing: Ночная обработка реестров, сканов документов, финансовых отчетов.



REST vs gRPC vs SSE

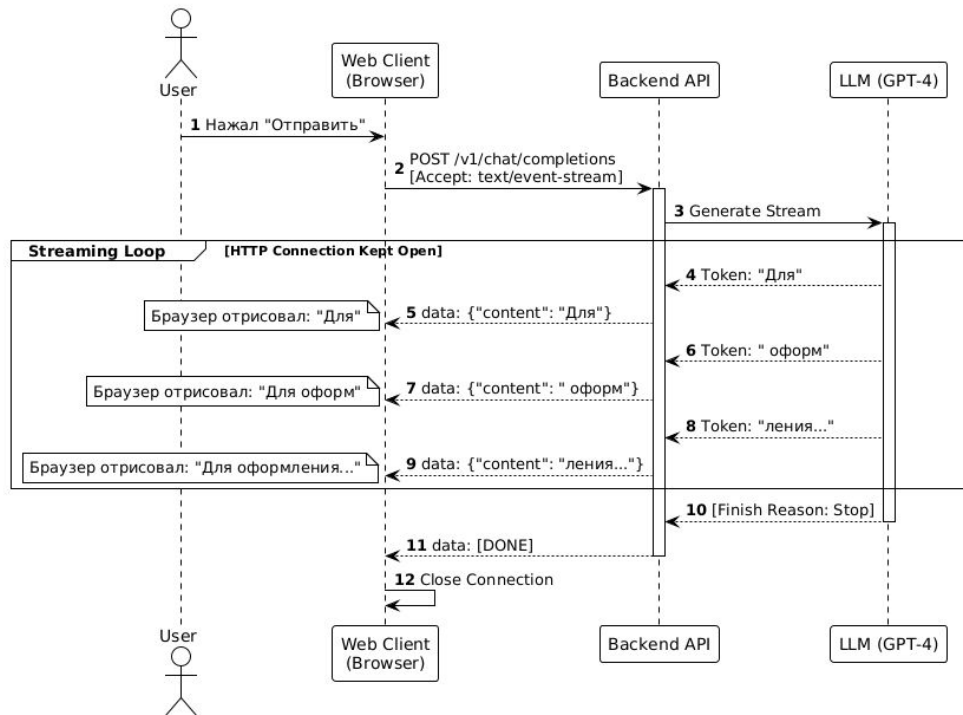
Протокол	REST (JSON)	gRPC (Protobuf)	SSE (Server-Sent Events)
Формат	Текст (JSON)	Бинарный (Proto)	Текст (Stream)
Почему	Прост в отладке (cURL)	Скорость	LLM возвращает токены
Эффективность	Низкая (оверхед на парсинг текста)	Высокая (компактный бинарный формат)	Средняя
Типизация	Слабая (OpenAPI как опция)	Строгая (.proto контракт)	Слабая
Streaming	Нет (или сложно)	Bi-directional (туда-обратно)	Unidirectional (сервер -> клиент)
Use Case в AI	Public API, Management	Internal (Backend <-> Model), Vector Data	LLM Token Streaming (Chat UX)



Паттерн LLM Token Streaming (SSE)

- Проблема: LLM генерирует текст. Ждать полного ответа долго.
- Server-Sent Events (стандарт HTML5):
 - Клиент открывает постоянное HTTP-соединение.
 - Сервер пушит чанки (токены) по мере генерации.
 - Браузер отрисовывает текст "на лету"
- Почему не WebSockets? Вебсокеты сложнее в балансировке и поддержке (stateful). SSE — это обычный HTTP, он проще и идеально подходит, когда говорит только сервер (LLM), а клиент слушает.

SSE: Потокковая генерация ответа (Streaming)



Паттерн API Gateway

- Скрывает сложность внутренней топологии от клиента. Клиент видит один хост `api.ai-bank.com`, а не десяток микросервисов.
- Снимаем с микросервисов рутину (SSL termination, Auth, Logging). В AI это важно, чтобы сервисы тратили ресурсы CPU/GPU только на бизнес-логику.
- Критично для AI. Внешний мир хочет JSON, внутренний мир гоняет тензоры по gRPC. Gateway делает конвертацию на лету.
- Первая линия обороны. Не пропускаем больше запросов, чем может переварить GPU-кластер.



Идемпотентность – это

свойство API, при котором повторный вызов с теми же параметрами не меняет состояние системы.

- Кейс: "Пользователь нажал кнопку 'Сгенерировать' в платном сервисе. Сеть моргнула, он нажал еще раз. С него списали деньги дважды за один промпт?"
- Реализация:
 - Клиент генерирует UUID (Idempotency-Key) и шлет его в заголовке.
 - Сервер проверяет в Redis: "Я уже видел этот ключ?".
 - Если видел — возвращает сохраненный результат без повторного запуска LLM.
- Важность для AI: Генерация дорогая (GPU time). Мы обязаны защищать ресурсы от дублей.



Паттерны устойчивости

- Тайм-ауты:
 - Ошибка: Дефолтный тайм-аут HTTP-клиента часто 30 сек. Для GPT-5 этого мало.
 - Решение: Ставим агрессивные тайм-ауты на connect (2 сек) и мягкие на read (60+ сек).
- Повторы с экспонентой (Exponential Backoff Retries):
 - Логика: Если получили 429 Too Many Requests, не долбим сервер сразу. Ждем 1с, потом 2с, 4с, 8с.
 - Jitter: Добавляем случайную задержку, чтобы все наши инстансы не пришли с повтором одновременно (Thundering Herd problem).
- Предохранитель (Circuit Breaker):
 - Если VectorDB упала, перестаем слать в неё запросы.
 - Принцип: Fail Fast. Сразу отдаем пользователю ошибку или кешированный ответ, давая базе время восстановиться.



Асинхронные протоколы и паттерны

Асинхронная обработка через синхронные протоколы

- Когда применять: Генерация отчетов, анализ больших PDF (время выполнения > 1 мин).
- Polling (Опрос):
 - Клиент шлет POST /analyze. Получает 202 Accepted и task_id.
 - Клиент каждые 5 сек шлет GET /tasks/{id} пока статус не станет COMPLETED.
 - Минус: Лишняя нагрузка на базу.
- Webhooks (Вебхуки):
 - Клиент передает callback_url.
 - Сервер сам "стучится" к клиенту с результатом, когда закончит.
 - Плюс: Идеально для Server-to-Server интеграций.



Паттерн "Отправил и забыл"

- Суть: Отправляем сообщение и не ждем подтверждения обработки. Максимальная скорость, минимальные гарантии.
- Где применять в AI:
 - Logging Prompts: Сбор всех промптов и ответов для дообучения (Fine-tuning). Если один лог потеряется — не страшно.
 - User Feedback: Лайки/дизлайки ответов модели.
- Риск: Потеря данных при сбое брокера. Не использовать для финансовых транзакций.



Паттерн “Очередь задач”

- Проблема: AI-инференс плохо масштабируется мгновенно (Cold start GPU долгий).
- Решение: Брокер как буфер.
 - В данном случае нам нужна Message Queue (RabbitMQ/SQS), где сообщение исчезает после обработки.
- Результат: Мы защищаем GPU от DDOS-атаки со стороны пользователей.



Паттерн "Издатель-Подписчик"

- Принцип: Один издатель — много подписчиков. Издатель не знает, кто его слушает.
- Слабая связанность: Мы можем добавить новый AI-сервис (например, "Анализ тональности"), не меняя код загрузчика файлов. Просто подписываемся на тот же топик.
- Отличие от Queue: В очереди сообщение читает один консьюмер. В Pub/Sub — все активные подписчики получают копию.



Выбор брокера (RabbitMQ vs Kafka)

Характеристика	RabbitMQ	Kafka
Модель	Очередь задач (Smart Broker)	Лог событий (Dumb Broker)
Доставка	Удаляется после прочтения (Ack)	Хранится N дней (Replay)
Сложность	Средняя	Высокая
Производительность	Средняя	Очень высокая
AI Use Case	Inference Queue (распределение задач на воркеры)	Training Data / Logs (сбор данных для обучения)



Паттерн "Dead Letter Queue"

- Проблема: Сообщение, которое крашит парсер или модель (например, битый PDF или атака на модель).
- Риск: Без DLQ такое сообщение будет вечно крутиться в цикле (Infinite Loop), забивая ресурсы.
- Решение: После N попыток изолируем сообщение в DLQ для ручного разбора.



ETL vs ELT: Как кормить модели данными

- ETL: Extract -> Transform (Clean) -> Load. Жесткая схема, потеря сырых данных.
- ELT: Extract -> Load (Data Lake) -> Transform. Гибкость.
- Почему ELT для AI:
 - Эксперименты с эмбедингами требуют пересчета данных.
 - Если исходники лежат в Data Lake, мы можем регенерировать вектора без повторной выгрузки из источника.
- Инструменты: Airflow, dbt, Dagster.



Интеграция с Legacy

- CDC (Change Data Capture)
 - Как узнать, что в старой базе что-то изменилось, не нагружая её запросами?
 - Читаем логи транзакций базы.
 - Превращаем изменения (Insert/Update/Delete) в поток событий.
 - AI Use Case: Синхронизация знаний. Клиент обновил адрес в монолите -> через секунду RAG-бот уже знает новый адрес.
- ACL (Anti-Corruption Layer)
 - Защитный слой. Не тяните старые, кривые названия полей в новую красивую AI-систему.
 - Конвертируем данные на входе в вашу доменную модель.



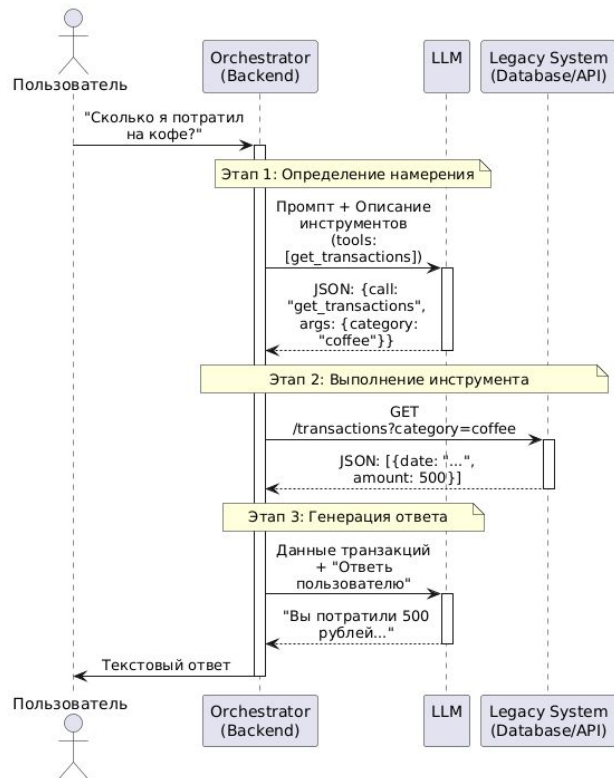
Практический кейс

Задача: Чат-бот с доступом к счетам

- Акторы:
 - User: "Сколько я потратил на кофе в этом месяце?"
 - LLM (Public Cloud): "Языковая модель не имеет доступа к вашим личным данным".
 - Legacy Core: Хранит транзакции, работает на COBOL/XML, очень медленный, доступ строго внутри контура.
- Риски:
 - Нельзя отдавать все транзакции в облако (Security/PII).
 - Нельзя заставлять пользователя ждать 20 секунд (Latency).
 - LLM может галлюцинировать цифры (Accuracy).



Как LLM ходит в API



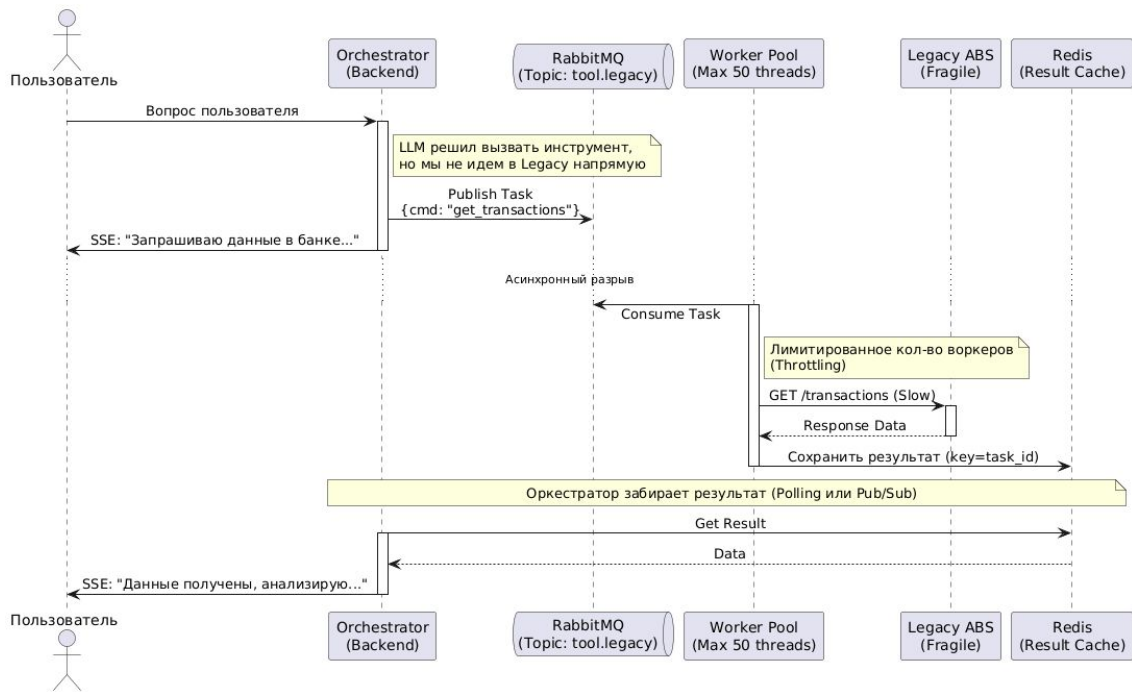
- Разделение ответственности
 - LLM отвечает за понимание намерения пользователя и извлечение параметров (сущностей). Не имеет доступа к сети/БД.
 - Orchestrator выполняет реальные вызовы API. Гарантирует безопасность и валидацию.
- Мы заставляем модель отвечать строго определенным JSON-схемой. Это устраняет необходимость парсить текст регулярками.
- Модель не придумывает цифры баланса, она получает их в контекст от надежного источника.

Проблема латентности и кэширование

- Legacy медленный: Выписка может строиться секунды.
- Если пользователь спросил "Траты на еду", а через минуту "Траты на рестораны" — это одни и те же транзакции. Кешируем сырой датасет выписки на 5-10 минут.
- UX: Используем стриминг (SSE), чтобы показывать промежуточные шаги ("Вызываю банк...", "Читаю ответ...", "Формирую отчет").



Асинхронное выполнение



- Нельзя блокировать поток генерации ожиданием HTTP-ответа.
- Throttling: Мы точно знаем, что АБС держит 50 запросов в секунду. Настраиваем ровно столько консьюмеров. Остальные задачи ждут в очереди.
- UX: Пользователь видит индикатор прогресса, а не зависший экран.

AI Security Gateway: Защита данных

- Правило: Никогда не отправлять PII (ФИО, полные номера карт) в публичные LLM (OpenAI/Anthropic).
 - Либо заменяем чувствительные данные на псевдонимы перед отправкой в облако.
 - Либо используем локальные модели (Llama 3 on-premise), где данные не покидают контур.



Разбор схемы отказа

- Проблема: Легаси система упала (500 Error) или отвечает дольше 30 секунд.
- Решение
 - Используем Circuit Breaker
 - Оркестратор видит рост ошибок и «выключает» инструмент `get_transactions` из доступных.
 - Мы не заставляем пользователя ждать тайм-аута.
 - Graceful Degradation (Изящная деградация):
 - Чат-бот не падает полностью. Он отключает только функции, зависящие от упавшего сервиса.
 - Общие вопросы (RAG по базе знаний, болталка) продолжают работать.



AI-стандарты и новые протоколы

Проблематика AI-интеграций

- Latency: Классический REST ждет 200 мс. LLM ждет 30 сек. HTTP-таймауты убивают соединения.
- Statefulness: В REST каждый запрос независим. В AI нужен контекст (History) — передавать его каждый раз дорого и долго.
- Non-Determinism: Один и тот же запрос дает разный ответ. Трудно тестировать и кешировать (нужен Semantic Cache).
- Hardware Bound: Масштабирование упирается не в CPU, а в наличие свободных GPU. Нельзя просто добавить подов в Kubernetes.



Уровень транспорта: JSON-RPC over HTTP/Stdio

- База всего: Большинство современных AI-протоколов (включая LSP в VS Code и MCP) работают поверх JSON-RPC.
- Почему JSON-RPC
 - Это stateful-сессия. В отличие от REST (stateless), здесь мы можем открыть канал и гонять сообщения туда-сюда («Вызови функцию» -> «Вот результат»/«Прогресс 50%»).
 - Stdio vs HTTP:
 - Local: Агент запускает процесс (например, python script.py) и общается через стандартный ввод/вывод (Stdio). Быстро, безопасно, но только локально.
 - Remote: Используем SSE (Server-Sent Events) для транспорта сообщений по сети.

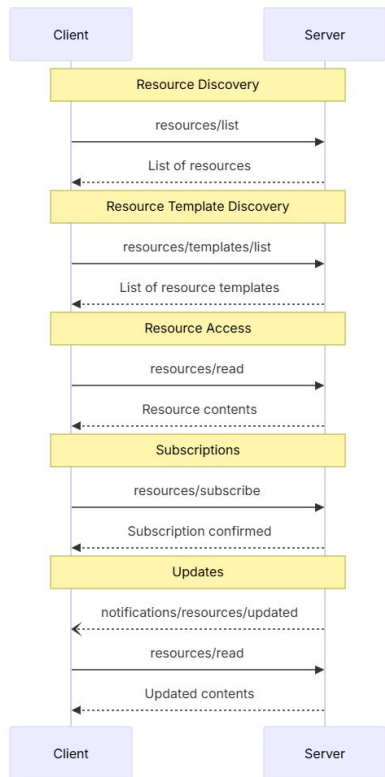
```
{  
  "jsonrpc": "2.0",  
  "method": "subtract",  
  "params": {  
    "minuend": 42,  
    "subtrahend": 23  
  },  
  "id": 3  
}
```

```
{  
  "jsonrpc": "2.0",  
  "result": 19,  
  "id": 3  
}
```



Уровень семантики: Model Context Protocol (MCP)

- Надстройка над JSON-RPC: MCP добавляет семантику. Он говорит: «Мы будем использовать JSON-RPC, но договоримся о фиксированном наборе методов».
- Универсальный драйвер:
 - Server: Оборачивает реальный источник (Postgres, Slack, FileSystem) в стандартные объекты MCP (Resources, Tools).
 - Client (LLM): Не знает специфики Postgres, но умеет вызывать tools/call.
- Профит: Замена NxM интеграций на 1xN.

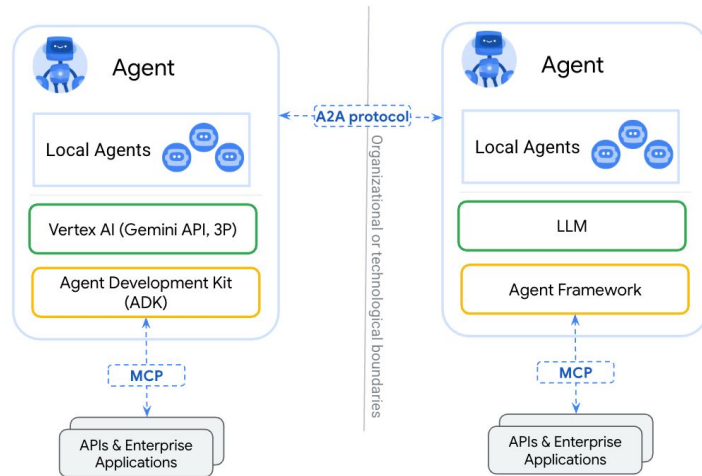


<https://modelcontextprotocol.io/docs/getting-started/intro>



Уровень поведения: Agent-to-Agent (A2A)

- Проблема MCP: MCP дает инструменты («Вот тебе молоток»), но не дает цели («Построй дом»).
- A2A (Agent-to-Agent): Это протокол социального взаимодействия.
 - Handshake: Агенты знакомятся («Я кодер», «Я менеджер»).
 - Objective: Передача высокоуровневой цели.
 - Delegation: Менеджер делегирует подзадачу, используя MCP-сервер другого агента как инструмент.
- Supervision: Протокол должен включать механизм прерывания (Stop sequence), если агент заиклился.



<https://a2a-protocol.org/>



ONNX: Единый стандарт обмена моделями

- Проблема: Data Scientists пишут на Python/PyTorch. Продакшн требует C++ или Go для скорости. Переписывать модель вручную — долго и чревато ошибками.
- Решение: ONNX (Open Neural Network Exchange). Это формат файла, который описывает нейросеть как граф вычислений, независимый от языка программирования.
- Вы можете обучить модель в PyTorch, экспортировать в .onnx и запустить её в высокопроизводительном Go-сервисе без установки Python.



**Подведём
итоги занятия**

Теперь вы сможете:

Отправьте в чат номера достигнутых целей

выбирать подходящие паттерны интеграции (API Gateway, Message Broker, ETL) для различных сценариев

проектировать обмен данными между AI-компонентами и традиционными системами

для того, чтобы AI компоненты не потеряли связь с окружающим миром



Вопросы для проверки

1. Почему для чат-бота с LLM протокол Server-Sent Events (SSE) подходит лучше, чем WebSockets или Polling?
2. Вы интегрируете AI-сервис с банковским монолитом. Монолит выдерживает только 10 запросов в секунду. Какой паттерн защитит монолит от перегрузки?
3. В чем ключевая разница между паттерном ETL и ELT при подготовке данных для RAG?



Ключевые тезисы занятия

1. AI меняет требования к транспорту: Из-за долгого инференса (секунды) классические HTTP-таймауты не работают.
2. Защищайте ресурсы: GPU — это дорого, а Legасу-системы — хрупко.
3. Разделяйте ответственность: LLM — это «мозг» (принятие решений), но не «руки».



Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️